Порядок обновления и физический цикл

Порядок обновления и физический цикл

Внутри игрового цикла вызывается `update()`. Но что **внутри** `update()`? Обработка ввода, ИИ, движение, коллизии, игровая логика, анимации, очистка. Порядок, в котором это выполняется, **определяет корректность игры**. Перепутали - объекты проходят сквозь стены, враги атакуют после смерти, физика ведёт себя по-разному на разных компьютерах.

Допустим, вы написали `update()` в том порядке, в каком пришло в голову:

```
cleanup()        // remove dead entities
ai_system()      // think - reads deleted entity -> crash
animation()      // animate - AI hasn't decided what to do yet
movement()       // move - no collision check after this!
```

Ну, пример утрированный, но суть ясна. Давайте разберём правильный порядок.

Типичный порядок систем

```
1. Input       - обработка ввода (клавиши, мышь)
2. AI          - решения ИИ (FSM, BT, minimax)
3. Movement    - применение скоростей, сил
4. Collision   - обнаружение столкновений
5. Resolution  - разрешение столкновений (отталкивание, урон)
6. Game Logic  - проверка условий (смерть, победа, триггеры)
7. Animation   - обновление кадров анимации
8. Cleanup     - удаление мёртвых сущностей
```

Почему так?

- **Input перед AI**: ИИ реагирует на текущее состояние мира - включая то, что игрок сделал в этом кадре. Логично, что сначала надо бы получить новый ввод игрока.
- **AI перед Movement**: ИИ решает бежать к игроку и устанавливает новую скорость. Movement применяет скорость -> сущность двигается. Сделаем наоборот - получим задержку в реакции.
- **Movement перед Collision**: Сначала все двигаются, потом проверяем, кто столкнулся. Если проверять коллизии между ходами - часть сущностей в старых позициях, часть в новых.

- **Resolution перед Game Logic**: Сначала отталкиваем из стены, потом проверяем "жив ли". Иначе игрок может умереть внутри стены.
- **Cleanup в конце**: Удаляем мёртвые сущности только после того, как все системы их обработали. Удалить раньше - и какая-то система получит обращение к несуществующему объекту.

Python C#

```python
def update(world, delta_time):
    input_system(world)
    ai_system(world, delta_time)
    movement_system(world, delta_time)
    collision_detection(world)
    collision_resolution(world)
    game_logic(world)
    animation_system(world, delta_time)
    cleanup_dead(world)


def draw(world, screen, camera):
    render_system(world, screen, camera)
```

Ок. Если вы используете ECS, то порядок систем - это буквально порядок вызовов в игровом цикле. В event-driven архитектуре порядок определяется тем, кто подписан на какие события и когда они вызываются. В обоих случаях проблемы те же.

Фиксированный шаг (Fixed Timestep)

deltaTime решает проблему скорости: движение выглядит одинаково на 30 и 120 FPS. Но **физика** с переменным шагом нестабильна. Проблема: при лаге `dt = 0.1` вместо шести шагов по 0.016 делается **один шаг в 6 раз длиннее**. Объект перескакивает стену за один кадр - коллизия пропущена. Прыжок получается разной высоты на разных FPS (гравитация интегрируется с разным шагом - результат отличается). Числа с плавающей точкой накапливают ошибку по-разному.

Решение: **физика обновляется с фиксированным шагом**, независимо от FPS.

Python C#

```python
FIXED_DT = 1.0 / 60.0  # physics at 60 Hz
accumulator = 0.0

while running:
    frame_time = clock.tick() / 1000.0  # actual frame time
    frame_time = min(frame_time, 0.25)  # cap: prevent spiral of death
    accumulator += frame_time
```

```
    # input and AI: once per frame (variable dt, not FIXED_DT)
    input_system(world)
    ai_system(world, frame_time)

    # physics: fixed step, may run 0, 1, or several times
    while accumulator >= FIXED_DT:
        movement_system(world, FIXED_DT)
        collision_detection(world)
        collision_resolution(world)
        accumulator -= FIXED_DT

    game_logic(world)
    animation_system(world, frame_time)
    cleanup_dead(world)
    draw(world, screen, camera)
```

Spiral of death: допустим, случился лаг в 500 мс. В накопитель добавляется 0.5 секунды - это 30 физических шагов по 0.016. Обсчитать 30 шагов тоже долго - кадр растягивается ещё сильнее. В следующий кадр накопитель ещё больше. Физика не успевает "расплатиться" с долгом, каждый кадр делает только хуже - игра зависает. Строка `frame_time = min(frame_time, 0.25)` ставит потолок: за один кадр накопитель получит максимум 0.25 с (≈15 шагов). Если реальный лаг был длиннее - остаток просто теряется. Физика замедлится, но не зависнет.

Идея - **накопитель**. Каждый кадр мы добавляем в него реальное время кадра. Как только накопилось на один физический шаг (`FIXED_DT`) - списываем и считаем физику. Если накопилось на два шага - считаем дважды. Если не накопилось - пропускаем.

```
FIXED_DT = 16 мс (60 Hz физика)

Кадр 1: реальный кадр = 33 мс (30 FPS)
  накопитель: 0 + 33 = 33 мс
  шаг 1: 33 - 16 = 17 мс (осталось)
  шаг 2: 17 - 16 = 1 мс  (осталось)
  → физика отработала 2 раза

Кадр 2: реальный кадр = 8 мс (120 FPS)
  накопитель: 1 + 8 = 9 мс
  9 < 16 → ждём
  → физика не вызвалась

Кадр 3: реальный кадр = 8 мс (120 FPS)
  накопитель: 9 + 8 = 17 мс
```

```
17 |   шаг 1: 17 - 16 = 1 мс
       → физика отработала 1 раз
```

FPS скачет - а физика всегда получает одинаковый `FIXED_DT = 16 мс`. Прыжок одинаковый на 30 FPS и на 144 FPS.

> MonoGame имеет встроенный `IsFixedTimeStep = true` с `TargetElapsedTime`. Но полезно понимать, как это работает изнутри - особенно если вы пишете на pygame, где фиксированного шага из коробки нет.

Интерполяция для плавности

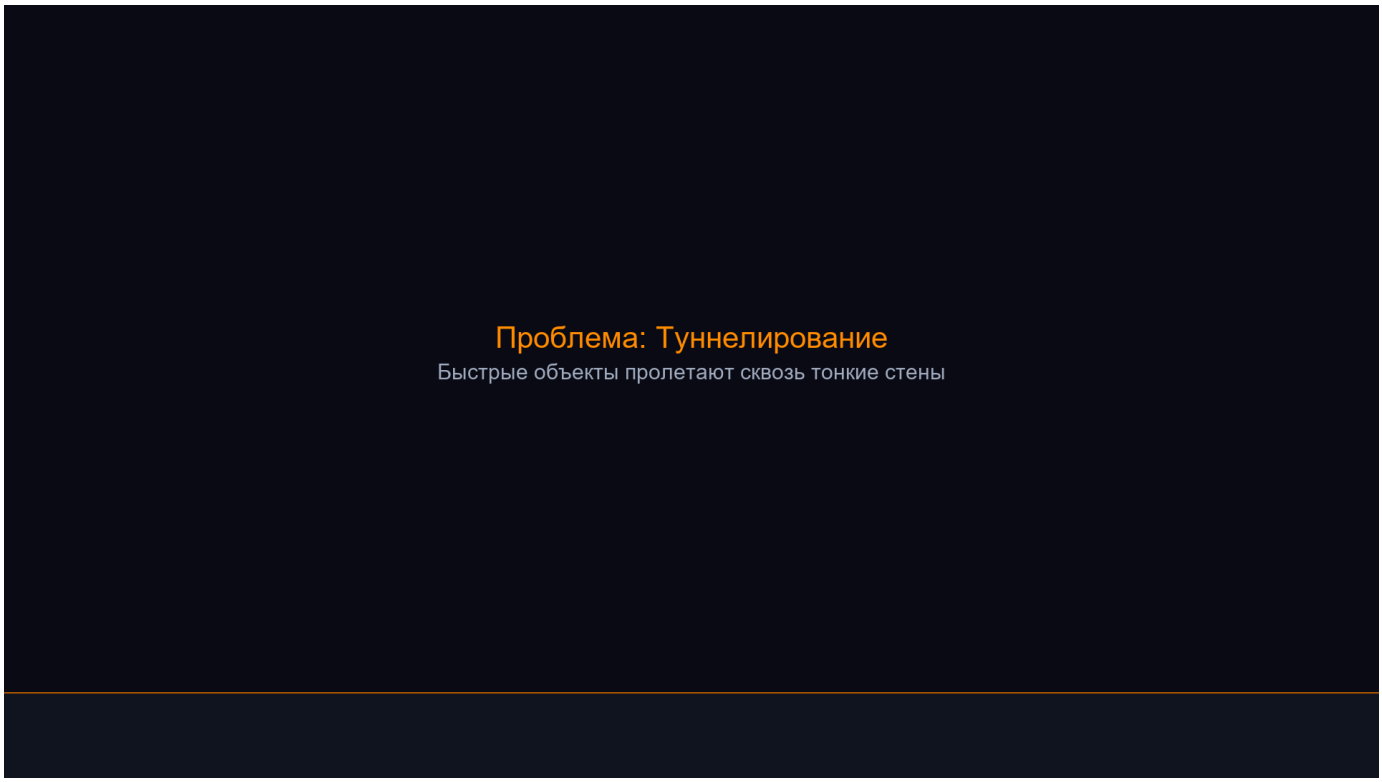
Хм. Есть побочный эффект: если рендер быстрее физики (120 FPS, физика 60 Hz), то между физическими обновлениями объекты "замирают" - рисуем одну и ту же позицию 2 кадра подряд. Решение - **интерполяция**:

Python | C#

```python
alpha = accumulator / FIXED_DT
render_x = prev_x + (current_x - prev_x) * alpha
render_y = prev_y + (current_y - prev_y) * alpha
```

Рисуем не текущую физическую позицию, а промежуточную между предыдущей и текущей. Это чисто визуальная коррекция - физика по-прежнему дискретна.

Туннелирование



Быстрый снаряд летит со скоростью 2000 пикселей в секунду. Стена толщиной 10 пикселей. При `dt = 0.016` снаряд за кадр пролетает 32 пикселя - он **перескакивает** стену. В позиции A (перед стеной) коллизии нет. В позиции B (за стеной) - тоже нет. Стену "не заметили".

```
Кадр N:    [снаряд]  |стена|
Кадр N+1:            |стена|  [снаряд]   ← прошёл насквозь
```

Решения

1. Swept collision (непрерывная проверка)

Вместо проверки "точка пересекает прямоугольник" - проверяем "отрезок от старой позиции до новой пересекает прямоугольник":

Python | C#

```python
def swept_aabb(old_x, old_y, new_x, new_y, wall):
    """Check if movement segment intersects wall AABB."""
    dx = new_x - old_x
    dy = new_y - old_y

    # parametric intersection: find t where segment enters/exits wall
    if dx != 0:
        t_enter_x = (wall.left - old_x) / dx
        t_exit_x = (wall.right - old_x) / dx
        if t_enter_x > t_exit_x:
            t_enter_x, t_exit_x = t_exit_x, t_enter_x
```

```python
    else:
        t_enter_x = float('-inf')
        t_exit_x = float('inf')

    if dy != 0:
        t_enter_y = (wall.top - old_y) / dy
        t_exit_y = (wall.bottom - old_y) / dy
        if t_enter_y > t_exit_y:
            t_enter_y, t_exit_y = t_exit_y, t_enter_y
    else:
        t_enter_y = float('-inf')
        t_exit_y = float('inf')

    t_enter = max(t_enter_x, t_enter_y)
    t_exit = min(t_exit_x, t_exit_y)

    if t_enter < t_exit and t_enter >= 0 and t_enter <= 1:
        return t_enter  # collision at this fraction of movement
    return None
```

Возвращает `t` - долю пути, на которой произошло столкновение. `t = 0.3` значит "столкнулся на 30% пути". Двигаем объект до `old + (new - old) * t`, а не до `new`.

Этот код проверяет **точку** против прямоугольника-стены. Если движущийся объект тоже имеет размер (AABB), увеличьте стену на половину размеров объекта перед проверкой. Иначе объект частично войдёт в стену до срабатывания коллизии.

2. Подшаги (substeps)

Проще, но дороже: разбиваем один большой шаг на несколько маленьких:

Python C#

```python
SUBSTEPS = 4
sub_dt = delta_time / SUBSTEPS
for _ in range(SUBSTEPS):
    move(entity, sub_dt)
    if check_collision(entity, walls):
        resolve(entity)
```

При 4 подшагах снаряд проходит 8 пикселей за подшаг вместо 32. Стена толщиной 10 - уже не проскочишь.

3. Ограничение максимальной скорости

Самое простое: `max_speed = wall_thickness / dt` . Если стены 10 пикселей и dt = 0.016, то max_speed ≈ 600. Грубо, но для многих игр достаточно.

Swept collision - правильный подход. Подшаги - запасной вариант. Ограничение скорости - костыль, но работающий.

Zombie-обновления

Враг получает урон -> `hp <= 0` -> враг "мёртв". Но его `update()` ещё не закончился - или он стоит в списке сущностей на обновление после текущей. Результат: мёртвый враг атакует, запускает анимацию, публикует событие.

```
enemy.hp -= 100    // enemy dies
...
enemy.update()     // still runs! attacks player from beyond the grave
```

Решение: флаг + отложенное удаление

Python | C#

```python
class Entity:
    def __init__(self):
        self.alive = True

    def kill(self):
        self.alive = False  # mark, don't remove yet


def ai_system(world, dt):
    for entity in world.entities:
        if not entity.alive:
            continue  # skip dead entities
        entity.ai.update(dt)


def cleanup_dead(world):
    world.entities = [e for e in world.entities if e.alive]
```

Правило: **каждая система проверяет** `alive` **перед обработкой**. Удаление - только в `cleanup` , в самом конце кадра. Так ни одна система не обратится к уже удалённому объекту.

Не удаляйте сущности из списка **во время итерации** по нему. В Python это пропустит элементы. В C# - выбросит `InvalidOperationException`. Собирайте "на удаление" отдельно, удаляйте после цикла.

Порядок разрешения столкновений

Допустим, A и B сталкиваются. Мы отталкиваем A от B. Но если B тоже движется и сталкивается с C - порядок обработки влияет на результат. Обработали A->B первым - A сдвинулся. Обработали B->C вторым - B сдвинулся, но A уже не пересчитывается.

При двух объектах это незаметно. При 50 - результат зависит от порядка в списке.

Решение: фазы

Разделяем обнаружение и разрешение:

```
Фаза 1 (Detection):
    Собираем ВСЕ пары столкновений в список
    [(A, B), (B, C), (D, E), ...]

Фаза 2 (Resolution):
    Для каждой пары: вычисляем вектор выталкивания
    Применяем ВСЕ выталкивания одновременно
```

Python | C#

```python
def collision_detection(world):
    """Phase 1: find all collisions."""
    world.collisions = []
    entities = world.entities
    for i in range(len(entities)):
        for j in range(i + 1, len(entities)):
            overlap = check_overlap(entities[i], entities[j])
            if overlap:
                world.collisions.append((entities[i], entities[j], overlap)


def collision_resolution(world):
    """Phase 2: resolve all at once."""
    for a, b, overlap in world.collisions:
        # split pushback equally
        a.x -= overlap.nx * overlap.depth * 0.5
        a.y -= overlap.ny * overlap.depth * 0.5
```

```
            b.x += overlap.nx * overlap.depth * 0.5
            b.y += overlap.ny * overlap.depth * 0.5
```

Сначала **все** столкновения обнаружены. Потом **все** разрешены. Порядок в списке сущностей больше не влияет на результат - все пары обрабатываются на основе одного и того же "снимка" позиций.

Это O(n²) - каждый с каждым. Для сотен объектов нужно пространственное разбиение: сетка, QuadTree. Они уменьшают число проверок, но фазовый подход (detection -> resolution) остаётся тем же.

Задержка на один кадр

Система A устанавливает значение. Система B читает его. Если B выполняется **раньше** A в порядке обновления - B прочитает значение **с прошлого кадра**.

```
Порядок A (AI -> Animation):
  AI:        enemy.wants_to_attack = True
  Animation: читает enemy.wants_to_attack -> True

Порядок B (Animation -> AI):
  Animation: читает enemy.wants_to_attack -> False (AI ещё не запустился)
  AI:        enemy.wants_to_attack = True
```

Решение - правильный порядок систем (AI перед Animation). Но иногда зависимости **циклические**: A зависит от B, B зависит от A. Тогда задержка на один кадр неизбежна.

При 60 FPS один кадр - 16 мс. Человек не заметит. Но если задержка накапливается (A -> B -> C -> D, каждый читает предыдущий с задержкой) - получаем 4 кадра (66 мс) между действием игрока и видимой реакцией. Это уже ощущается.

Как отладить: замедление (`time_scale = 0.1`) делает задержку видимой. Если при замедлении анимация запускается заметно позже действия - у вас проблема с порядком.

Сводка: что может пойти не так

Проблема	Причина	Решение
Объект проходит сквозь стену	Туннелирование: шаг > толщины стены	Swept collision, подшаги, max speed
Физика зависит от FPS	Переменный `delta_time`	Фиксированный шаг с аккумулятором

Проблема	Причина	Решение
Мёртвый враг атакует	Удаление во время обработки	Флаг `alive` + cleanup в конце
Коллизии зависят от порядка в списке	Resolution во время detection	Фазы: сначала detect all, потом resolve all
Анимация запаздывает	AI обновляется после Animation	Правильный порядок систем
Прыжок разной высоты на разных ПК	`delta_time` влияет на физику нелинейно	Фиксированный шаг
Дёрганье при высоком FPS	Физика реже рендера	Интерполяция позиций

Итого

Что	Где подробнее
Игровой цикл и deltaTime	Основной цикл
Обнаружение столкновений (AABB, тайлы)	Коллизии
Оптимизация коллизий (сетка, QuadTree)	Пространственные структуры
Системы в ECS и их порядок	ECS и EC
ИИ как система (FSM, BT)	ИИ врагов, деревья поведений
Событийная модель для межсистемного общения	Событийная модель
Анимации и их обновление	Анимации
Визуализация порядка обновления	Отладка
Типичные ошибки (логика в Draw, нет deltaTime)	Типичные ошибки

Powered by Wiki.js